



Introduction To Linux Operating System

General Introduction

Prof. Badre BOUSALEM



Plan du cours

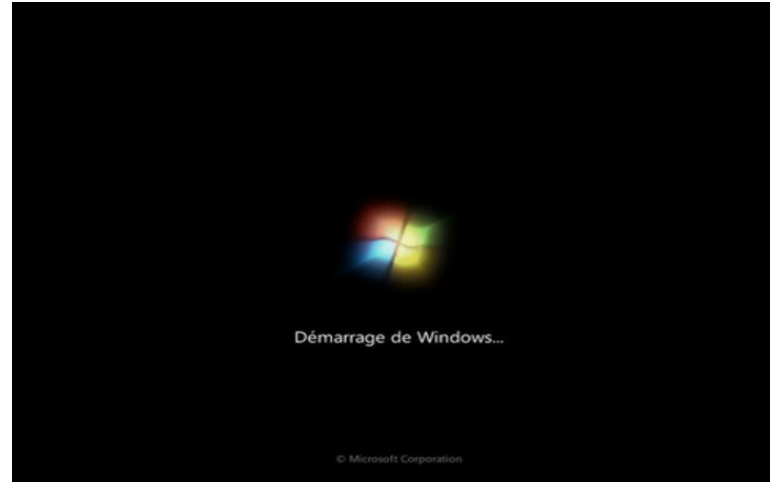
- I. History of UNIX development
- II. Shell
- III. Shell variables
- IV. The command line
- V. Log In and Log Out
- VI. Password
- VII. User and group information
- VIII. Man pages

I. History of UNIX development



What is an operating system?(1/2)

When starting a computer, the first thing that appears is a screen like this one:



→ This screen may change depending on the version of Windows or another operating system (example: MAC)

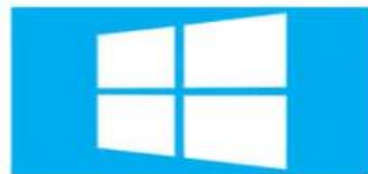
**Once Windows is loaded, you can use your programs:
games, Internet, drawing software, mail, music...**

What is an operating system?(2/2)

- ❑ An operating system, or system software, is a software in an electronic device that drives hardware devices and receives instructions from the user or from other software or applications.
- ❑ In a computer, the operating system manages the processor(s) and the memory. It operates the peripherals (keyboard, mouse, screen, hard disk, motherboard...)
- ❑ Operating systems also include the interface with the user. In a computer, for example, it will display the windows and present the contents of the storage units (hard disk, CD, DVD...).

Examples of OS:

Windows



MAC



Linux



Unix

- UNIX (1969 by Ken Thomson, MIT): The oldest computer operating system
 - Management of resources, processor, memory and peripherals
- **Features:**
 - **Multi-tasking:** several programs can run simultaneously
 - **Multi-user:** several users can work at the same time on the same machine

GNU Project

Project GNU

Launched by [Richard Stallman](#) (1984)

Objective : create a free and complete operating system

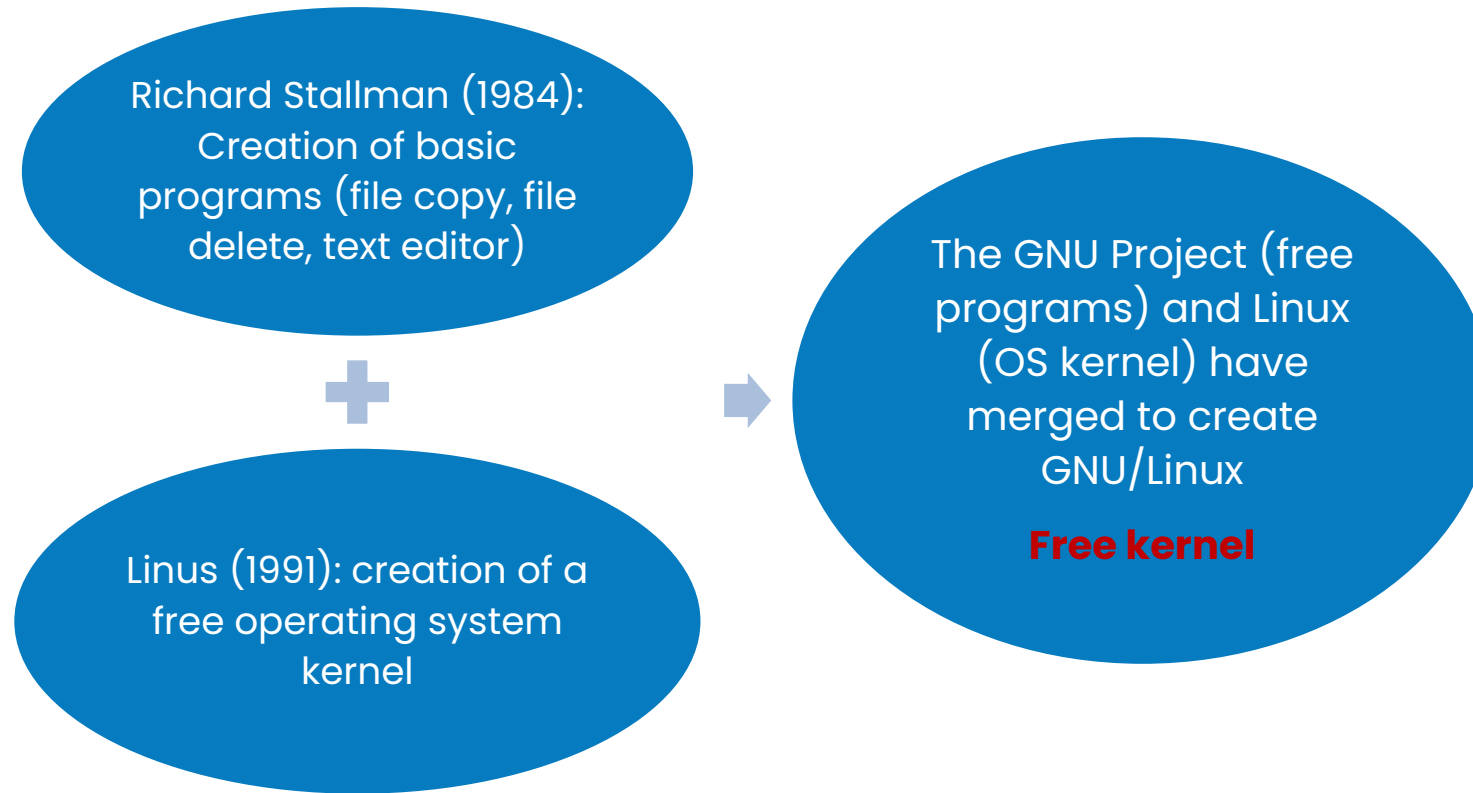
GNU : Gnu's Not Unix (recursive acronym)

- + Offers the necessary free software (C compiler (gcc), make (GNU make), Emacs,
- C library (glibc), basic tools (ls, cp ...)

☹ **But it lacks the kernel**

GNU/Linux

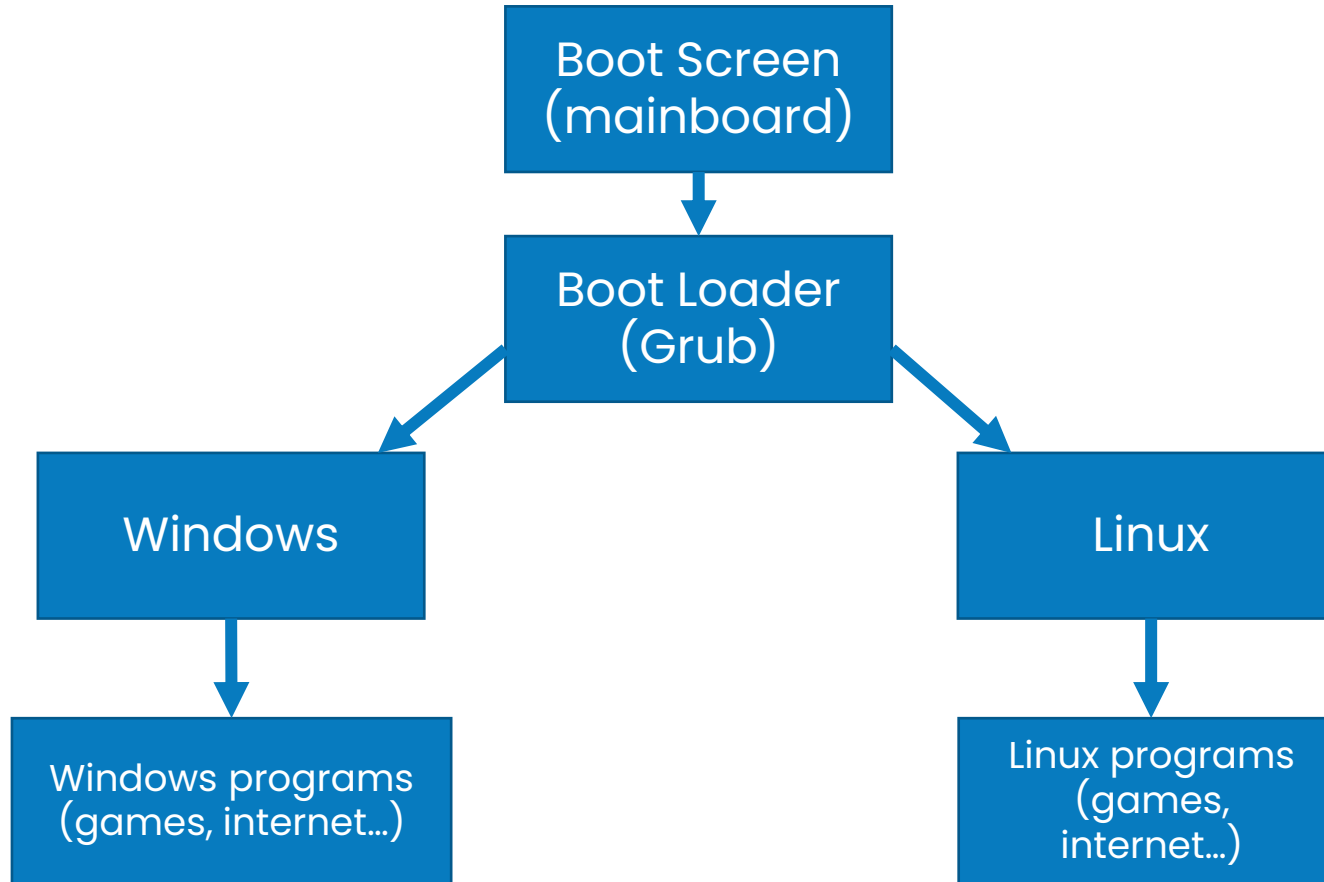
- **Linux:** Launched in 1991 by Linus Torvalds (Helsinki University, 1991)



→ Linux (Linus + Unix) [originally called Freax = free+freak+unix].

Linux

Linux is an operating system, just like Windows or Mac OS



The main differences between Windows and Linux

Windows

- A commercial operating system
 - Source code is inaccessible
 - Is not customizable
- Must be booted from the main partition
 - File names are not case sensitive
- Uses the microkernel which takes up less space but the efficiency of the system is lower than Linux
- Windows has a rich graphical interface and can be easily used by technical and non-technical people.
→ It is very simple and user-friendly.

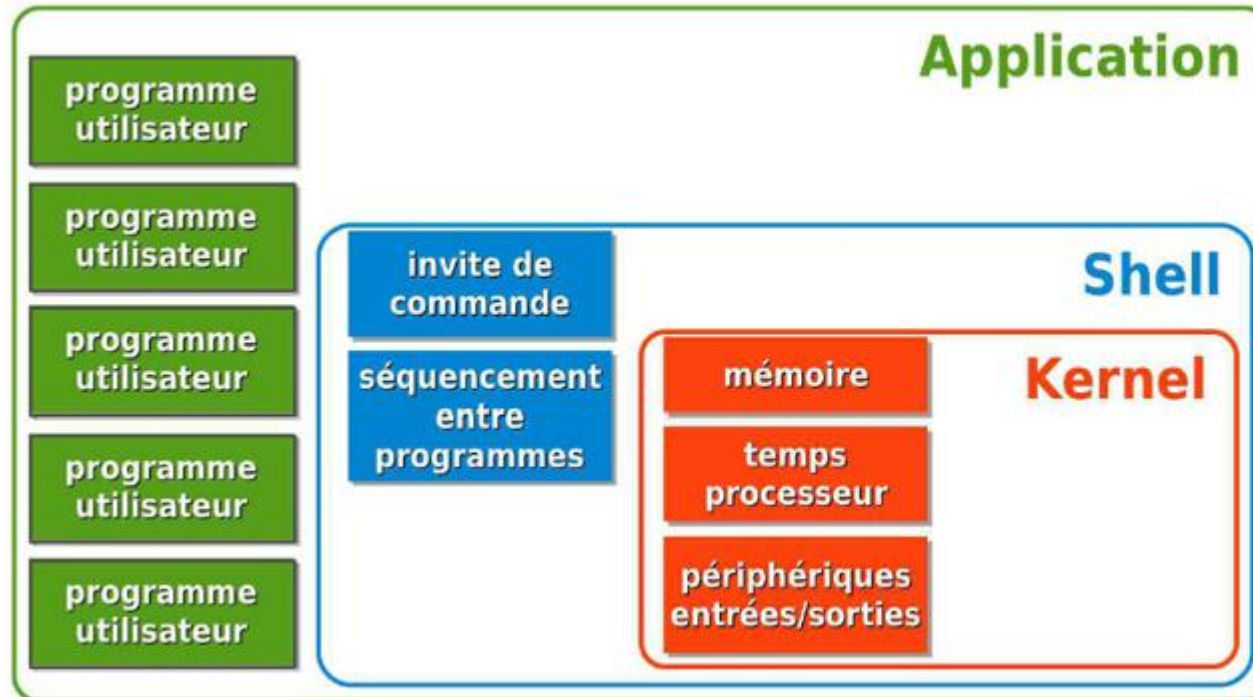
Linux

- A free operating system
 - The source code is open
- Customizable, a user can change the code and its appearance
- Provides high security compared to Windows because Linux is open source
 - Can be booted from the main or logical partition
 - File names are case sensitive
- Uses the monolithic kernel which consumes more working space (the whole operating system runs in the kernel space).
- It is mainly used by technicians because you need to know various Linux commands to work with the Linux operating system.
- The maintenance process in Linux is more complicated than in Windows.

What are the advantages of having Linux?

- ✓ **Accessibility:** Linux is an open-source operating system. Its source code is freely available and can be exploited by developers who wish to adapt it to their needs.
- ✓ **Stability:** A number of distributions offer a stabilized application ecosystem. FreeBSD or Debian (to a lesser extent) have a robust and approved software catalog. They rarely offer extremely recent tools that have not been approved by the community in terms of stability.
- ✓ **Security:** The security of this operating system is recognized and inherent to its architecture. The management of rights, users and their structure makes the security of this system powerful.
- ✓ **Performance:** Most of the performance is based on the quality of the drivers that allow the hardware to communicate with the operating system.
- ✓ **Customization:** Depending on the desktop-oriented Linux distribution, you will find a number of pre-installed applications as standard. Server versions (without a graphical interface) usually come with the minimum amount of software. You can then decide to install or uninstall as many non-vital functions as you wish.

GNU/Linux system



Free software: Four freedoms

- ✓ Freedom 0: The freedom to run the program
- ✓ Freedom 1: The freedom to study how the program works and to adapt it
=> (access to source code required)
- ✓ Freedom 2: The freedom to redistribute copies
- ✓ Freedom 3: The freedom to improve the program and to publish its improvements for the benefit of the entire community.
=> (access to source code required)

Linux distribution

- Linux is a very rich operating system.
 - There are hundreds of different ways to install it, different distributions of Linux have been created:
 - The concept of distribution is unique to Linux (does not exist in windows, similar to the difference between Windows7 home and Windows 7 professional) The difference from one distribution to another is in:
 1. The installation: very simplified as very complicated
 2. The management of the programs installation: If it is well done and centralized, it can make the installation of new software easier than under Windows.
 3. the programs pre-installed on the computer (for example, Windows comes with Internet Explorer and Windows Media Player).
- **A distribution is in a way the packaging of Linux. The core remains the same on all distributions.**
- **No matter which distribution you install, you get a Linux that is compatible with the others. Some distributions are just more or less easy to get used to.**

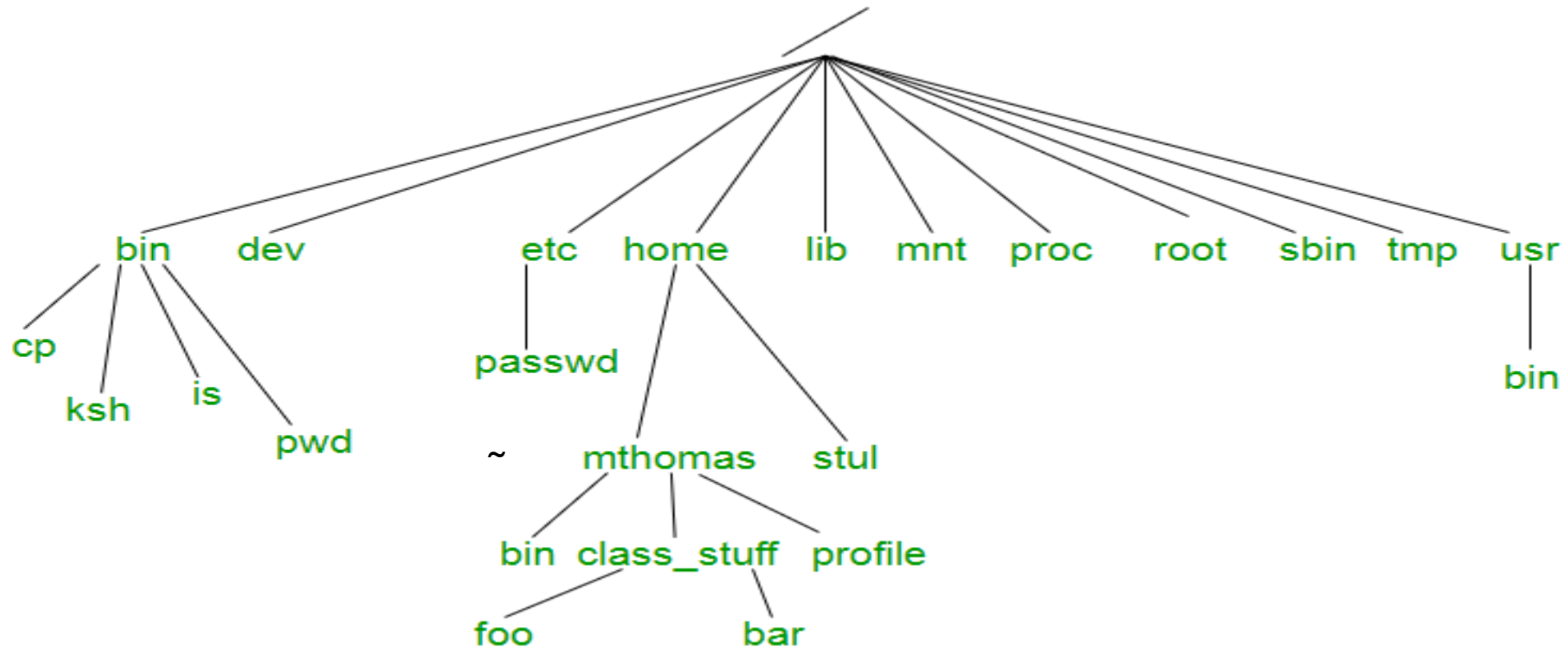
Choice of Distribution

	Maintainer distribution	Public	Oriented	Packages
RedHat 	Company (RHEL)	Advanced (Beginner)	Server	RPM
*Fedora 	Community	Advanced (Beginner)	Workstation / Server	RPM
*CentOS 	Community	Advanced (Beginner)	Workstation / Server	RPM
SuSE/Novell 	Enterprise Community	Advanced (Beginner)	Server	RPM
Debian 	Community	Advanced	Workstation / Server	DEB
*ubuntu 	Community	Beginner	Workstation / Server	DEB
*LinuxMint	Community	Beginner	Workstation	DEB

Linux Certification



Linux structure directory



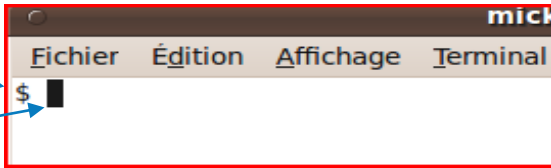
II. Shell



Shell

- Le shell : "the shell around the kernel"
 - User interface-Linux core
 - A command interpreter
 - A powerful programming language => Writing scripts
- The shell is as follows

prompt
Cursor



Prompt : Message displayed to indicate that the system is awaiting command entry

The character \$ is displayed to designate the prompt of a normal user :

\$ user-command-entry

The character # is displayed to designate the administrator prompt:

administrator-command-entry

Do not type the prompt string when testing commands

Different shells (1/2)

Differents shells, among others :

- **sh**: Bourne Shell under unix
=> Korn shell (ksh), improved version by AT&T
- **Bash**: Bourne Again Shell, enhanced version GNU
- List of shells in the system : **cat /etc/shells**
- The shell launched at startup is the one associated with the login **/etc/passwd**
- Choose another shell for a user

chsh -s /bin/sh nomutilisateur

Exemple: change the shell of the user Mickey from bash to sh

```
zouhour@zouhour-virtual-machine:~$ chsh -s /bin/sh Mickey
```

- End shell session **\$ exit**

Different shells (2/2)

Shell	Path	Default Prompt (non-root user)	Default Prompt (Root user)
The Bourne Shell (sh)	/bin/sh and /sbin/sh	\$	#
The C Shell (csh)	/bin/csh	%	#
The Korn Shell (ksh)	/bin/ksh	\$	#
The GNU Bourne-Again Shell (Bash)	/bin/bash	bash-x.xx\$	bash-x.xx#

SHELL `/bin/bash` (1/3)

The shell program `/bin/bash` uses a set of startup files to help create an environment. Each file has a specific use and may relate differently to the login and interactive environments:

- The shell call reads the file `/etc/profile`.
- An interactive non-login shell normally starts from the command line using a shell program (like `[prompt]$/bin/bash`)
- In the shell `bash`, when the user logs on, he runs the general configuration `/etc/bash.bashrc`. This can define terminal colors, aliases, etc.
- The user can also customize his configuration through the `~/.bashrc` file, which allows him to change the terminal colors, create his own aliases or modify the `PATH`.

SHELL /bin/bash (2/3)

- A non-interactive shell is usually present when a shell script is running. It is non-interactive because it executes a script and does not wait for user input between commands. For these Shell calls, only the environment inherited from the parent Shell is used.
- The \$HOME/.bash_logout file is read and executed when a user leaves an interactive login Shell.

Many distributions use /etc/bashrc for initialization for the entire non-login shell system. This file is usually called from the user's \$HOME/.bashrc file and is not built directly into bash itself.

SHELL /bin/bash (3/3)

- The history of the commands typed is in the file: `.bash_history`
 - A script is a sequence of commands and operations executed in a shell.
 - Scripts executed by bash manage among other things the initialization of environment variables.
- 

How the Shell works: summary

✓ **The command ends with carriage return**

✓ **The shell**

Substitutes variables

Identifies the words

Interprets wildcards to generate filenames (shell expansion)

Deletes the quote characters

Substitutes nested commands

Syntax	Expression Value	Expansion	Explication
{ }	echo ba{ba,bu}	Baba babu	enumeration
\	echo \"\ ' \\$ \{ \} \A B	" '\$ { A B	emission
\$	echo \$HOME	/home/roger	Variable value
\${...}	echo \${HOME}	/home/roger	Variable value
\$(...)	echo \$(which true)	/bin/true	Command output
\$((...))	echo \$((12+4*2))	20	Expression output

III. Shell variables



Variables

- A variable is a storage space for a result.
- A variable can change its type during its existence. This is called weak typing.
- It is not the variables that have a type, but the values. This is called dynamic typing.
- A variable exists as soon as it is assigned a value (by default the type will be a string). An empty string is a valid value.
- By default, variables are local to the shell. Variables exist only for the current shell. If you want to make them accessible to other shells, you have to export them with the export command (for bash), or use the setenv command (csh).
- There are also environment variables which are dynamic variables used by the different processes of an operating system (Windows, Unix, etc.).
They are usually defined in CAPITALS :
 - \$PATH under Unix/Linux
 - %PATH% under Windows

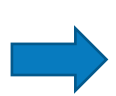
Using Variables in the Shell

➤ Shell variables (local):

- Specific to the Shells in which they are defined
- In lower case (by convention)

➤ Environment variables (global) :

- They constitute the execution environment for the programs.
- Predefined in upper case (by convention)
- Known to all commands launched from the Shell
- Inherited at the sub-Shell level



***To create an environment variable, you must define it and export it
After modifying an environment variable, it must be exported***

Shell variables

All local variables in a bash session can be viewed with `set` and all variables in the current environment can be viewed with `env`.

- To declare a local variable:

```
[root@test ] /# VARNAME=SOMETHING
```

- To remove a variable

```
[root@test ] /# unset VARNAME
```

- To declare a variable that must be seen by all shells

```
[root@test ] /# export VARNAME=SOMETHING
```

or

```
[root@test ] /# VARNAME=SOMETHING
```

```
[root@test ] /# export VARNAME
```

Manipulation of shell variables

- **Definition:**

`variable=value`

- **Content:**

`$variable`

`${variable}`

- **delete:**

`unset variable`

- **Environnement Variables :**

`variable=value`

`export variable`

Or

`variable=value`

`declare -x variable`

Or

`export variable=value`

- **Display:**

`echo $variable`

Quotes

- **\$var**

Substitute a variable with its value

- **Simple Quote '**

Does not interpret special characters (copy)

```
$ echo 'Jha is $username'
```

- **Backquotes `**

Used to substitute a command with its result

- Double quote "**

Allows the substitution of variables (preceded by \$ and the commands (between ``) without interpreting other characters.

```
$ echo « Welcome $USER"
```

```
$echo "it's `date`"
```

- **Backstroke) **

Does not allow the interpretation of the following character

Examples

Example 1:

```
$ var="test string"  
$ newvar="Value of var is $var"  
$ echo $newvar  
Value of var is test string
```

Example 2:

```
$ var='test string'  
$ newvar='Value of var is $var'  
$ echo $newvar  
Value of var is $var
```

Some environment variables

- List of paths to directories containing programs **PATH**
- To execute a command, specify its path : \$ **/opt/bin/command**
- To run it without specifying its path, add it to the PATH

```
$ PATH=${PATH}:/opt/bin  
$ PATH="$PATH:/opt/bin" } $ export PATH  
$ PATH=$PATH:/opt/bin
```

HOME Path to the user's home directory (**cd**)

PS1 The primary shell prompt (by default \$) **\$PS1="\u@\h:\w\\$"**

PS2 The secondary shell prompt (by default >)

✓ **\t** time

✓ **\d** date

✓ **\w** current directory path

✓ **\h** nameMachine

✓ **\W** current directory

✓ **\u** username

✓ **\\$** prompt character

✓ if UID ≠ 0, \$ displayed else #

✓ **\!** N°historical

IV. Command Line



Command Line

All command lines have this syntax :

Command [options] [arguments]

Examples:

```
[root@test ] /# pwd
[root@test ] /# ls -ld ; ls -l -d ; ls -d -l
[root@test ] /# rm -r /tmp/toto
[root@test ] /# cat ../readme helpme > save
[root@test ] /# more /etc/passwd /etc/hosts /etc/group
[root@test ] /# find . -name *.[ch] -print
[root@test ] /# date "+day is %a"
```

Command repetition

- `!!` repeat previous command
 - `!n` : repeat nth command
 - `!-n` : repeats nth command from end
 - `history`
- 

Command sequences

cmd1;cmd2

Sequential execution of cmd1 and cmd2 commands

cmd1&&cmd2

Execution of cmd2 if cmd1 executed without error

cmd1||cmd2

Running cmd2 if cmd1 returns an error

cmd1&

Allows the execution of a command in the background

`cmd` ⇔ \$(cmd)

V. Log In et Log Out



Log In et Log Out

- The log in session is the first step to access a Linux system.
- A username (login ID) and a password are required to use the system.
- The root login ID is the login ID of the super user who has all the privileges normally assigned to the system administrator. It is not possible to log in with the root user on Ubuntu because this account is disabled. You have to log in as a normal user and execute commands with **sudo**.
- Once a user has logged in, the environment variables load and start the shell that corresponds to the user.
- Some variables loaded at log in:

\$HOME is the user's home directory.

\$SHELL is the shell used

\$PATH is a list of directories separated by colons. These directories are the ones where the shell will look for commands that the user can run in

- The log out uses the **exit** or **logout** command.

VI. Password



Password

Under Linux, each user has a password (except root under Ubuntu which has no default password).

To change passwords of users or groups, type the passwd command.

passwd [options] username/group

Some options :

- l: To block an account.

- g|l: used for groups. g to set a password for a group and r to delete it.

- x|n: Maximum and minimum number of days a password is valid.

- u: Unblock an account.

VII. User and group information



User information(1/4)

- 3 user types:

owner (user)

people from the same group (group)

others (others)

- 3 permission types:

	File	directory
Read (r)	Show content	Show content
Whrite (w)	modify	Create/delete files
Execution (x)	Execute	Cross

User information (2/4)

- root
A particular user (superuser) has full root privileges
Reserved for installation and administration of the system
For security reasons, we avoid connecting to root, use sudo
To create user : useradd
- Users are identified by UIDs, an integer on the interval 0 .. 4 294 967 295 (FFFFFFFF). Users with UID=0 are super-users
- Users belong to groups identified by GIDs, an integer on the interval 0 .. 4 294 967 295
- adduser and addgroup can automatically choose UIDs and GIDs
- id displays your identification :

```
uid=1000(vg) gid=1000(vg),groupes=4(adm),1000(vg)
```

User information (3/4)

- In the `/etc/passwd` file, one line defines each user account. The encrypted password is in `/etc/shadow`.
- The format of `/etc/passwd` is as follows:

```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/bin/bash
daemon:x:2:2:Daemon:/sbin:/bin/bash
rarrigoni:x:500:100:Raphael Arrigoni:/home/rarrigoni:/bin/bash
```

1. The first field is the username
2. The x field represents the old password location.
3. The third and fourth fields are the UID and GID of the user
4. The fifth field is a comment
5. The sixth field is the user's home directory.
6. The last field is the user's default shell

User information (4/4)

The format of **/etc/shadow** is as follows:

```
root:IMXweyiV816gg:11996:0:10000::::  
bin:!:8902:0:10000::::  
daemon:*:8902:0:10000::::  
rarrigoni:GoIM8jlS.IuTY:11996:0:99999:7:::
```

1. Username.
2. 13-character encrypted password, a blank entry means that there is no need to provide a password (dangerous!), the ! in front of the encrypted password means that the account is blocked. The * means that no password has been set yet.
3. Number of days (since January 1, 1970) since the password was changed
4. Number of days before the password can be changed (0 means that the password can be changed at any time)
5. Number of days after which the password must be changed (enter 99999 to mean that the user can keep his password indefinitely)
6. Number of days to notify the user of a password about to expire (enter 7 for a full week)
7. The number of days after expiration, during which the account is still active
8. The number of days (since January 1, 1970) that the account has been deactivated.
9. Field reserved for future use

Group Information (1/2)

In the `/etc/group` file, there is a line for each user group. Encrypted passwords are in `/etc/gshadow`.

The format of `/etc/group` is as follows:

```
root:x:0:root
bin:x:1:root,bin,daemon
daemon:x:2:
video:x:33:rarrigoni
```

1. The first field is the username
2. The second field x means that the group password is encrypted in the `/etc/gshadow` file.
3. The third field is the GID
4. The fourth field contains the members of the group

Group Information (2/2)

The format of `/etc/gshadow` is as follows:

```
root::root:root
bin::root:root daemon::root:root
bin:!*:8902:0:10000:::
video::root:root,rarrigoni
```

1. The first field is the name of the group
2. The second field is the encrypted password.
3. The third field is a list of group administrators.
4. The fourth field is a list of members of the group

Passwords for groups are usually not activated until all group members have received the password from the administrator.

VIII. Man pages



Man pages

The online manuals describe most of the commands used in your system

man [command]

```
[root@test ] /# man mkdir  
[root@test ] /# man cal
```

Convenient controls of the man .

- / : search text
- n: go to the following occurrence
- N: go to the previous occurrence
- q: get out